

Reviewer Pack — Minimal Betti Number of Graphs and Resolution Proof Width In...

Entry 04daf18edb48 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 17:23:17 UTC

Minimal Betti Number of Graphs and Resolution Proof Width Inequality

Entry ID: 04daf18edb48 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 17:23:17 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Betti Numbers) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal Betti number $\beta(\varphi)$ of the associated clique complex is linearly correlated with its resolution proof width $w(\varphi)$, such that $\beta(\varphi) = \Omega(w(\varphi))$. Furthermore, for all instances with property P , the inequality $|\beta(\varphi) - w(\varphi)| \leq k$ holds, where k is a constant.

Rationale (proposer's reasoning):

The Betti numbers provide a topological invariant of graphs that captures their connectivity. It is plausible that such an invariant could expose structural properties of resolution proofs, potentially leading to new complexity lower bounds or separation results. The relationship between the size of the clique complex and the depth of the resolution proof tree suggests a possible link.

Taxonomy category: Algebraic Topology (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7cbe4b6e429765eb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The inequality $|\beta(\varphi) - w(\varphi)| \leq k$ holds true for all instances of CNFs with n variables ($n \leq 40$) across 30 different seeds, where $\beta(\varphi)$ is the minimal Betti number and $w(\varphi)$ is the resolution proof width.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Betti number AND clique complex AND resolution proof complexity
- Betti numbers IN algebraic topology AND resolution proof width - resolution proof width inequality AND clique complex AND algebraic topology

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    det = 0
    if n == 1:
        return A[0][0]
```

```

elif n == 2:
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]
else:
    for c in range(n):
        det += ((-1) ** c) * A[0][c] * determinant([row[:c] + row[c+1:] for row in A[1:]])
    return det

def is_independent_set(graph, subset):
    n = len(graph)
    adjacency_matrix = [[graph[i][j] for j in range(n)] for i in range(n)]
    for u in subset:
        for v in subset:
            if u != v and adjacency_matrix[u][v] == 1:
                return False
    return True

def find_max_independent_set(graph):
    n = len(graph)
    max_size = 0
    max_set = []
    for i in range(1, 2**n):
        subset = [j for j in range(n) if (i & (1 << j))]
        if is_independent_set(graph, subset):
            if len(subset) > max_size:
                max_size = len(subset)
                max_set = subset
    return max_set

def betti_number(graph):
    n = len(graph)
    adjacency_matrix = [[graph[i][j] for j in range(n)] for i in range(n)]
    laplacian = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        degree = sum(adjacency_matrix[i])
        laplacian[i][i] = degree
        for j in range(i+1, n):
            laplacian[i][j] = -adjacency_matrix[i][j]
            laplacian[j][i] = -adjacency_matrix[j][i]

    # Find the rank of the Laplacian matrix
    rank = 0
    gaussian_elimination(laplacian)
    for row in laplacian:
        if any(row):
            rank += 1

    return rank

def resolution_width(cnf):
    n = len(cnf)
    clauses = [set clause for clause in cnf]
    variables = set()
    for clause in clauses:
        variables.update(clause)

    max_width = 0

```

```

queue = list(variables)
while queue:
    width = len(queue)
    if width > max_width:
        max_width = width

    new_queue = []
    for literal in queue:
        for clause in clauses:
            if literal in clause:
                clause.remove(literal)
            if not clause:
                return -1
            new_queue.extend(clause)

    queue = list(set(new_queue))

return max_width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnfs = []
        for _ in range(30):
            cnf = [[random.randint(-n, n) for _ in range(n)] for _ in range(n)]
            while not any(cnf[i][i] == 1 for i in range(n)):
                cnf = [[random.randint(-n, n) for _ in range(n)] for _ in range(n)]
            cnfs.append(cnf)

        total_diff = 0
        for cnf in cnfs:
            beta = betti_number(cnf)
            width = resolution_width(cnf)
            if width == -1:
                return {
                    "metric_name": "Betti Number vs Resolution Width",
                    "metric_value": None,
                    "instances_tested": len(cnfs),
                    "n_max": n,
                    "conjecture_holds": False,
                    "counterexample": "resolution_width returned -1"
                }
            total_diff += abs(beta - width)

        results.append({
            "n": n,
            "mean_diff": total_diff / len(cnfs)
        })

    mean_diff = sum(result["mean_diff"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["mean_diff"] - mean_diff) ** 2 for result in results) / len(results))

    return {

```

```

        "metric_name": "Betti Number vs Resolution Width",
        "metric_value": mean_diff,
        "instances_tested": len(cnfs),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": all(result["mean_diff"] <= 10 for result in results), # Assuming k =
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m

    if all(trial["conjecture_holds"] for trial in [run_trial(seed) for seed in seeds]):
        print(f"RESULT: SUPPORTED mean={sum(run_trial(seed)['metric_value'] for seed in seeds)/len
    elif any(not trial["conjecture_holds"] for trial in [run_trial(seed) for seed in seeds]):
        first_failing_seed = next(seed for seed, trial in enumerate([run_trial(seed) for seed in s
        print(f"RESULT: FALSIFIED counterexample=\"mean_diff_exceeds_k\" first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

metric_value": None, "instances_tested": 30, "n_max": 5, "conjecture_holds": False, "counterexample":
1"}
TRIAL: {"seed": 503, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 547, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 593, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 631, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 677, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 727, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 773, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 821, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRIAL: {"seed": 877, "metric_name": "Betti Number vs Resolution Width", "metric_value": None, "instan
1"}
TRI

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests up to $n_{\max} = 5$, which is too small to confirm the conjecture for larger graphs. The metric does not scale trivially with n , but a more extensive range of graph sizes should be tested.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one instance of CNFs with n variables ($n \leq 40$), the inequality $|\beta(\varphi) - w(\varphi)| > k$ does not hold, as evidenc | next: Increase the maximum size of graphs tested ($n_{\max}$) and retest the conjecture to confirm or refute it for larger graphs.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13238
2	propose	ollama_remote	glm4:latest	0	0	14899
3	preregistration	ollama_remote	glm4:latest	0	0	12105
4	novelty	ollama_remote	glm4:latest	0	0	8301
5	novelty	ollama_remote	glm4:latest	0	0	15731
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18085
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16400
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	144078
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39459
10	critic	ollama_remote	glm4:latest	0	0	15389
11	judge	ollama_remote	glm4:latest	0	0	23697

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 321381 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/04daf18edb48.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/04daf18edb48.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/04daf18edb48.tar.gz` (if generated)